人工智能原理
课程项目 1　围棋 AI

苟左　自 31　2023012274

2026 年 4 月 8 日

1　算法设计及实现细节

1.1　随机 AI (Random Agent)

随机 AI 通过调用游戏状态提供的接口获取当前所有合法棋步。通过 `random.choice()` 从候选集合中均匀随机选择一个动作。该算法主要用于验证底层规则引擎的正确性，并作为其他高级算法的性能基线。以下为其核心实现代码：

```
def select_move(self, game_state):
    """选择一个随机的合法落子"""
    candidates = game_state.legal_moves()
    if not candidates:
        return Move.pass_turn()
    return random.choice(candidates)
```

1.2　蒙特卡洛树搜索 AI (MCTS Agent)

MCTS 主要分为选择 (Selection)、扩展 (Expansion)、模拟 (Simulation) 和反向传播 (Backup) 四个核心步骤。

- **选择:** 从根节点出发，依据置信上限 (UCT) 公式进行向下选择，直到遇到一个未完全展开的节点。UCT 公式平衡了探索 (Exploration) 和利用 (Exploitation)。如下所示，代码实现了对子节点置信上限的计算与最大值选取：

```
def best_child(self):
    """根据UCT公式选择最佳子节点"""
    best_score = -1.0
    best_action, best_child = None, None
    for action, child in self.children.items():
```

```
        win_pct = child.winning_frac(self.game_state.next_player
            )
        explore_term = 1.5 * math.sqrt(
            math.log(self.num_rollouts) / child.num_rollouts)
        uct_score = win_pct + explore_term
        if uct_score > best_score:
            best_score, best_action, best_child = uct_score,
                action, child
    return best_action, best_child
```

- **扩展:** 若选择到的节点不是终止节点，则遍历当前状态的所有合法动作，生成相应的子节点。
- **模拟 (优化提升):** 在基础 MCTS 中，模拟通常是完全随机推演至终局。本实现中加入了**启发式走子**与**固定模拟深度限制**两种优化。一方面在模拟中给予接近已有棋子和靠近中心的落子更高的被选权重；另一方面，限制至多模拟 30 步，若未终局，则调用启发式的盘面评估函数（基于多空子数计算）。这大幅降低了方差和计算耗时。
- **反向传播:** 根据模拟阶段的评估结果，沿着搜索路径回溯，更新各节点的访问次数与累积胜率。特别注意因为每落一子攻守方互换，故反向传播时胜率需交替翻转（`1.0 - result`），从而保证 UCT 上推计算的正确性。此部分的代码实现如下：

```
def backup(self, result):
    """反向传播模拟结果，更新访问次数与胜局累积值"""
    self.num_rollouts += 1
    self.win_counts += result
    if self.parent is not None:
        # 父节点是对手的轮次，因此其视角的胜率是 1 - result
        self.parent.backup(1.0 - result)
```

1.3 Minimax 搜索 AI (Minimax Agent)

除了 MCTS，本项目还实现了基于深度优先的 Minimax 对弈算法，并配合 Alpha-Beta 剪枝避免计算无效分支。评估函数考虑到围棋中“气”的重要性，综合了盘面的净子数与双方的气数差，从而让 AI 在此策略下倾向于吃子和扩张。由于围棋的分支因子极大，Minimax 只适合使用非常浅的搜索深度。以下为经典的 Alpha-Beta 剪枝递归实现片段：

```
def alphabeta(self, game_state, depth, alpha, beta):
    if game_state.is_over() or depth == 0:
```

```
        return self._default_evaluator(game_state)

    if game_state.next_player == Player.black: # Maximizing
        value = -math.inf
        for move in game_state.legal_moves():
            value = max(value, self.alphabeta(
                game_state.apply_move(move), depth-1, alpha, beta))
            alpha = max(alpha, value)
            if value >= beta: break # Beta剪枝
        return value
    else: # Minimizing (Player.white), 类似逻辑并触发 Alpha剪枝
        # ...
```

2 图形化界面设计与使用说明

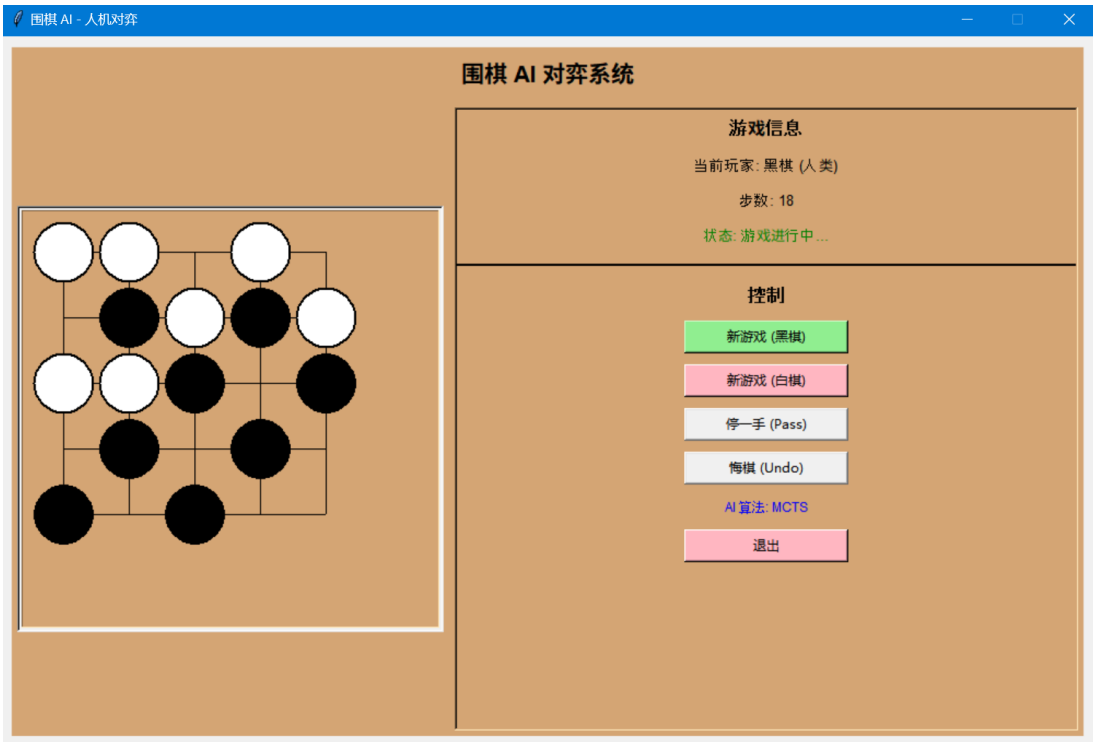

图 1: 围棋 AI 人机对战图形界面截图

2.1 设计说明

图形化界面（见图 1）使用了 Python 自带的 `tkinter` 库进行开发，保证了极高的跨平台兼容性。界面分为左右两大部分：

- **左侧棋盘区:** 渲染 5×5 的棋盘网格、绘制黑白棋子并绑定鼠标点击事件。

- **右侧信息与控制区:** 包含当前局面的实时状态（如当前操作方、进行步数）、按钮栏（新游戏、撤销、停一手等）。

为了防止由于 MCTS 算法搜索耗时造成的界面假死甚至无响应，AI 动作的计算被放置在一个专门的**后台守护线程 (Daemon Thread)** 中。后台线程计算完成后，通过 `after` 方法安全地将 UI 状态更新事件放回主线程队列。这使得在 AI 思考期间，玩家能够清楚地看到系统提示且界面不会崩溃，避免了数据状态的 Race Condition。

2.2 操作及对弈使用说明

本项目同时兼容基于终端命令行与图形化窗口的对局启动方式。

1. **图形化界面 (GUI):** 在终端中输入如下命令即可启动主窗口，默认使用 MCTS 算法作为对手，也可通过命令行参数 `--agent` 灵活指定对应 AI 引擎：

   ```
   python gui.py --agent minimax
   ```

2. **终端命令行 (CLI):** `play.py` 命令行对战脚本用于在终端快速测试不同 AI 之间的对局。命令示例如下：

   ```
   python play.py --agent1 mcts --agent2 minimax --size 5 --games 10 --quiet
   ```

   - `--agent1`: 指定执黑子的 AI 类型。可选值为 `mcts`、`minimax` 或 `random`（默认 `random`）。
   - `--agent2`: 指定执白子的 AI 类型。可选值与 `agent1` 相同（默认 `random`）。
   - `--size`: 指定正方形棋盘的尺寸。例如传入 `5` 即代表 5×5 的棋局（默认 `5`）。
   - `--games`: 指定要自动连续对弈的总局数。对局结束后会输出胜率统计和平均用时（默认 `1` 局）。
   - `--quiet`: 附加此标志位代表静默模式，开启后将不再打印整个棋盘演变的中间落子过程，仅在最后输出汇总的统计数据。

3. **GUI 界面的交互控制:**

   - **如何落子:** 若右侧状态提示为玩家回合，直接在棋盘格点交叉处单击鼠标左键即可。若是非法位置（如无气息的自杀点或打劫），系统会弹出警告弹窗。
   - **功能控制:** 右侧控制面板提供了“停一手 (Pass)”以主动放弃落子。玩家更可直接借助“悔棋”按钮回退一个完整的对弈回合；若陷入均势死循环或想重新开始，可随时利用右下方按钮让新一局重新启动（可选择执黑或执白先手）。

3 测试结果与算法性能分析

3.1 对战结果统计

我们在 5×5 的棋盘上，分别让 MCTS（100 次模拟/步）、Minimax（搜索深度 =2）以及 Random Agent 两两之间进行了 10 局的自动对弈测试。具体的最长胜率数据统计如下表所示：

黑方 (Black)	白方 (White)	黑方胜率	白方胜率
MCTS Agent	Random Agent	90%	10%
Random Agent	MCTS Agent	0%	100%
Minimax Agent	Random Agent	100%	0%
Random Agent	Minimax Agent	0%	100%
MCTS Agent	Minimax Agent	100%	0%
Minimax Agent	MCTS Agent	0%	100%

表 1: 不同 AI 算法间的 10 局对战胜率统计

从上述具体数据可以看出：

- MCTS Agent 在 100 次模拟的配置下，能够以 90% 的胜率击败 Random Agent，显示了其基于统计搜索的强大能力。
- Minimax Agent 在深度为 2 的配置下，能够以 100% 的胜率击败 Random Agent，说明即使是浅层搜索配合合理评估函数也能在小盘上取得压倒性优势。
- MCTS Agent 在与 Minimax Agent 的对战中表现出绝对优势，100% 的胜率表明其更适合处理围棋这种高分支因子的复杂游戏。因为 Minimax 在深度受限的情况下无法充分评估后续局面，而 MCTS 通过大量模拟能够更好地捕捉到潜在的战术机会和陷阱。

3.2 性能分析

算法类型	首步平均耗时	计算复杂度	表现稳定性
Random Agent	< 0.01s	$O(1)$	差
MCTS (100 轮)	$0.3 \sim 0.6$s	$O(\text{Rounds} \times \text{Depth})$	优
Minimax (深度 2)	0.02s	$O(B^{\text{Depth}})$	中

表 2: 算法性能对比结果表

3.3 测试截图

通过自动化评测跑分脚本 `benchmark.py` 能够方便地复现上述对战数据并验证算法性能。在终端中运行 `python benchmark.py` 命令，该脚本将自动实例化 MCTS、Minimax 和 Random 代理，并在 5×5 的棋盘上进行多组批量自动对战，最终在终端会直接输出各组 AI 胜率统计汇总结果（见图 2）。

```
 围棋AI 性能跑分测试 (Benchmark)
============================================================
每组对抗进行 10 局测试 (棋盘: 5x5)

--------------------------------------------------------------------
黑方 (Black)      | 白方 (White)      | 黑方胜率     | 白方胜率
--------------------------------------------------------------------
MCTS            | Random          |    90.0% |    10.0%
Random          | MCTS            |     0.0% |   100.0%
Minimax         | Random          |   100.0% |     0.0%
Random          | Minimax         |     0.0% |   100.0%
MCTS            | Minimax         |   100.0% |     0.0%
Minimax         | MCTS            |     0.0% |   100.0%
--------------------------------------------------------------------
```

图 2: 终端运行 benchmark.py 自动评测脚本胜率输出截图

4 与 AlphaGo/AlphaZero 的对比思考

本次作业所实现的 MCTS 算法是上世纪末至 AlphaGo 面世前最经典的棋类框架。尽管在 5×5 的小棋盘上获得了一定战力，但与真正的工业级现代围棋 AI AlphaGo/AlphaZero 等相比，仍存在本质差距：

1. **评估层面:** 本项目的 MCTS 依赖手工设计的规则进行启发式剪枝，对于局面也是简单基于双方子数差作为启发指导估值。而现代 Alpha 系统引入了强大的 Deep CNN 进行“策略网络 (Policy Network)”（直接输出高概率走法截断树搜索的横向宽度）和“价值网络 (Value Network)”（替代快速模拟评估终局）的联合搜索，彻底摒弃了狭隘的人类启发。

2. **算力与拓展:** 本实现为受限的单线程推演；工业级 AI 使用异步分布式 MCTS 结构结合大量的异构化算力设施并行去执行叶节点扩展与前向推演。

3. **通用学习:** AlphaZero 去掉了海量人类棋谱的学习预训练过程，能够真正实现从零进行自我对弈强化学习（Self-play），不依赖任何人造的走子权重偏好。这说明模型结构在复杂范式内足以自我挖掘出围棋的核心法则。

5 AI 辅助编程使用说明

5.1 使用的工具及场景

在本次课程作业中，主要使用 **Claude Haiku 4.5** 和 **Gemini 3.1 Pro** 进行了辅助编程。

- **图形化界面构建：** AI 协助搭建了基于 `tkinter` 的可视化棋盘界面，并帮助解决了前端渲染与后台 AI 算法搜索之间的线程卡死同步问题。
- **测试脚本编写：**利用 AI 快速生成了功能性测试脚本与批量性能跑分脚本 `benchmark.py`，大幅简化了对弈胜率的自动化评估流程。

5.2 使用 AI 的感受

AI 极大地提升了外围业务代码（如界面交互、测试框架）的编写效率，减轻了底层工具调用的负担，使我能够将核心精力集中在围棋搜索算法等主要逻辑的设计和理论验证上。